
7-Day Sprint Plan

Agentic Text-to-SQL SLM — Learn + Build in One Week

This plan is designed for a **learn-by-building** approach. Each day has a morning learning block (1-2 hours of concepts, videos, docs) and an afternoon build block (4-6 hours of hands-on coding). By the end of each day, you have something working that you can demo.

Day	Theme	Learning Focus	Build Deliverable
1	Foundation + Schema Ingestion	SLMs, text-to-SQL basics, SQLAlchemy, pandas	Excel importer + SQL connector working
2	Graph Schema + Vector Store	Neo4j/NetworkX, graph traversal, FAISS	Graph built + join path discovery working
3	RAG Pipeline	Embeddings, retrieval, reranking, prompt design	End-to-end RAG retrieval returning schema context
4	Fine-Tuning the SLM	LoRA, QLoRA, PEFT, HuggingFace TRL	Fine-tuned model generating SQL on your schema
5	Agentic Layer	LangGraph, ReAct, agents, tool calling, MCP	Self-correcting agent with 3-retry loop
6	Validation + API + UI	sqlglot, FastAPI, Streamlit, Docker	Full app: chat UI → SQL output
7	Evaluation + Polish + Demo	Accuracy metrics, testing, presentation skills	Demo-ready system + evaluation report

Daily schedule: Morning (1-2 hrs) = Learn concepts. Afternoon (4-6 hrs) = Build. Evening (30 min) = Review + commit code + write notes for mentor.

DAY 1 Foundation + Schema Ingestion

Learn the basics, build the data pipeline

9:00 - 11:00

LEARN

Concepts & Background

- **What is Text-to-SQL?** — Read the Spider dataset paper abstract. Understand: NL input → SQL output.
- **What is an SLM?** — Small Language Models (1-7B params) vs LLMs. Why CodeLlama-7B and Phi-3-mini.
- **Watch:** 'Text-to-SQL with LLMs' by Yannic Kilcher or similar (YouTube, 20 min).
- **SQLAlchemy basics:** Read docs on engine, inspector, metadata reflection (30 min).
- **Understand the architecture:** Review your capstone PDF Sections 1-3. Trace the data flow

11:00 - 18:00

BUILD

Schema Ingestion Pipeline

- **Set up project:** Create Git repo, virtual environment, requirements.txt with pandas, openpyxl, sqlalchemy.
- **Create a sample Excel schema file** with 5-6 tables (users, orders, products, order_items, categories, reviews) — include all columns, PKs, FKs, descriptions.
- **Build Excel importer:** Python function that reads the Excel file → returns list of TableSchema dataclass objects with validated relationships.
- **Build SQL connector:** Function using SQLAlchemy inspector to connect to a SQLite/PostgreSQL sample DB and extract the same schema info programmatically.
- **Build schema normalizer:** Convert both sources into canonical CREATE TABLE DDL strings with FK annotations.
- **Test:** Import from Excel AND from a live SQLite DB — both should produce identical normalized

Day 1 — End-of-Day Checklist

- Can import schema from Excel (.xlsx) file
- Can connect to a SQL database and extract schema automatically
- Both sources produce the same canonical DDL format
- Created a sample database with 5-6 tables and realistic data (100+ rows)
- Code committed to Git with README
- Can explain to mentor: what is text-to-SQL, why SLMs, what the pipeline looks like

Tip: Create a good sample database on Day 1 — you'll use it every subsequent day. Include realistic table names, 5-6 tables with FK chains 3+ hops deep, and 100+ rows of data.

DAY 2 **Graph Schema + Vector Store**

Model relationships, enable join discovery

9:00 - 11:00

LEARN

Graph Databases & Embeddings

- **Graph basics:** Nodes, edges, properties. Why graphs for schema relationships (15 min).
- **NetworkX tutorial:** Create a graph, add nodes/edges, run BFS/shortest_path (30 min). Start with NetworkX — it's simpler than Neo4j and sufficient for your demo.
- **Neo4j concepts:** Read Cypher basics — MATCH, WHERE, shortestPath(). Install Neo4j Desktop or use Docker (optional, can use NetworkX instead).
- **Sentence embeddings:** What are vector embeddings? How does all-MiniLM-L6-v2 work? (15 min read).
- **FAISS basics:** What is approximate nearest neighbor search? IndexFlatIP vs IndexIVFFlat (15 min read).

11:00 - 18:00

BUILD

Graph Builder + Vector Index

- **Build graph schema builder:** Take normalized DDL from Day 1 → create NetworkX DiGraph. Tables = nodes (with column list, PK, description as attributes). FKs = directed edges (with from_col, to_col, cardinality).
- **Build join path finder:** Function: given two table names, use nx.shortest_path() to return the complete JOIN chain with all ON conditions. Test: find_join_path('users', 'categories') should return the 4-hop chain.
- **Handle multi-path:** When multiple paths exist, return all paths sorted by hop count.
- **Build FAISS vector index:** pip install sentence-transformers faiss-cpu. Embed each table's description text. Build FAISS IndexFlatIP. Function: given NL query, return top-k most similar tables.
- **Test both:** Query 'revenue by product category' → vector search returns [orders, products,

Day 2 — End-of-Day Checklist

- NetworkX graph built from normalized schema with all tables and FK edges
- find_join_path('users', 'products') returns correct 3-hop JOIN chain
- FAISS index built with table embeddings
- Vector search for 'total sales by category' returns relevant tables
- Both retrieval methods (graph + vector) produce consistent results
- Can explain to mentor: why graph for joins, why vector search for table retrieval

DAY 3 RAG Pipeline

Retrieve the right context at query time

9:00 - 11:00

LEARN

RAG Architecture & Prompt Engineering

- **RAG concept:** Why can't we just put the entire schema in the prompt? Token limits. Retrieval solves this. Read LangChain RAG tutorial (20 min).
- **Cross-encoder reranking:** Bi-encoder (fast, less accurate) vs cross-encoder (slower, more accurate). Why rerank after initial retrieval? (15 min).
- **Prompt engineering for SQL:** Study the prompt format: schema context + join hints + question → SQL. Look at how DIN-SQL and DAIL-SQL structure their prompts (20 min).
- **Context window math:** CodeLlama-7B has 4K tokens. Schema context ~2K, question ~100, SQL

11:00 - 18:00

BUILD

Full RAG Retrieval Pipeline

- **Build NL preprocessor:** Simple entity extraction — match words in the query against table/column names and business glossary. No need for spaCy yet — regex + fuzzy match (thefuzz library) works for the demo.
- **Build dual retriever:** Function that takes NL query → runs vector search (FAISS) + graph join planner (NetworkX) in parallel → returns combined context.
- **Build context assembler:** Merge retrieved table DDLs + join paths + column metadata into a structured prompt string. Enforce token budget (count with tiktoken).
- **Build prompt template:** '[INST] Given schema: {ddl}\nJoin path: {joins}\nQuestion: {question}\nGenerate SQL: [/INST]'
- **Test end-to-end:** NL question → preprocessor → dual retrieval → assembled prompt. Print the

Day 3 — End-of-Day Checklist

- NL preprocessor extracts table/column references from natural language
- Dual retriever combines vector search + graph join planning
- Context assembler produces well-structured prompts under token budget
- Tested with 5+ diverse NL questions — prompts look correct
- Can explain to mentor: what RAG is, why dual retrieval, how the prompt is constructed

DAY 4

Fine-Tuning the SLM

Make the model expert at your schema

9:00 - 11:00

LEARN

LoRA, QLoRA & HuggingFace PEFT

- **LoRA paper key idea:** Instead of updating all 7B parameters, add small trainable adapter matrices (rank 16 = ~10M params). Dramatically reduces VRAM. (20 min).
- **QLoRA:** Quantize base model to 4-bit (NF4) → attach LoRA adapters → train only adapters. 28GB → 8GB VRAM. (15 min).
- **HuggingFace ecosystem:** transformers (model loading), peft (LoRA), trl (SFTTrainer), bitsandbytes (quantization). How they fit together. (20 min).
- **Training data format:** Study the chat template for CodeLlama-Instruct or Phi-3. Your data must

11:00 - 18:00

BUILD

Generate Data + Fine-Tune

- **Generate training data:** Write a script that walks random paths (1-4 hops) in your graph, generates JOIN queries, then creates NL questions using templates: 'Show me {agg} of {column} by {group_col}' etc. Target: 500-1000 pairs (sufficient for demo).
- **Option A — Fine-tune on Colab/Kaggle:** Upload data, use QLoRA + SFTTrainer to fine-tune Phi-3-mini (3.8B, fits T4 GPU). Train for 1-2 epochs (~1-2 hours).
- **Option B — No GPU available:** Use a pre-trained code model (CodeLlama-7B-Instruct-GGUF) via llama-cpp-python in few-shot mode. Skip fine-tuning, use your RAG context as few-shot examples instead. This still works well!
- **Test generation:** Load the model (fine-tuned or few-shot) and test with your RAG prompts from Day 3. Does it produce valid SQL?
- **Save artifacts:** Export LoRA adapters (Option A) or save your few-shot prompt templates (Option

Day 4 — End-of-Day Checklist

- Generated 500+ NL-SQL training pairs from your schema graph
- Either: fine-tuned model producing SQL OR: few-shot setup producing SQL
- Model generates valid SQL for at least 5 out of 10 test queries
- Can explain to mentor: what LoRA/QLoRA is, why fine-tune, the training data format

Tip: Don't get stuck on fine-tuning if GPU is an issue. Option B (few-shot with a pre-trained model) is a perfectly valid approach for a capstone demo — the architecture is what matters, not whether you trained for 3 epochs vs used few-shot prompting.

DAY 5

Agentic Layer

Add reasoning, tools, and self-correction

9:00 - 11:00

LEARN

Agents, LangGraph & ReAct

- **What are AI agents?** — Agent = LLM + Tools + Memory + Reasoning loop. Read Anthropic's 'Building Effective Agents' blog post (20 min).
- **ReAct pattern:** Reason → Act → Observe → Repeat. The agent thinks about what to do, calls a tool, observes the result, and decides next step. (15 min).
- **LangGraph tutorial:** Follow the official quickstart — create a simple graph with nodes and edges. Understand: StateGraph, add_node, add_conditional_edges. (30 min).

11:00 - 18:00

BUILD

LangGraph Agent with Self-Correction

- **Define agent state:** TypedDict with: query, schema_context, join_paths, generated_sql, validation_result, error_msg, retry_count.
- **Build 3 agent nodes:** (1) planner_node — calls NL preprocessor + graph planner; (2) generator_node — assembles RAG prompt + runs SLM; (3) validator_node — checks SQL syntax with sqlglot.
- **Wire the graph:** START → planner → generator → validator → conditional edge: if valid → END, if invalid AND retry < 3 → generator (with error context appended to prompt).
- **Build tools:** Wrap your Day 2/3 functions as LangGraph tools: schema_graph_tool, rag_retrieval_tool, sql_generator_tool.
- **Test the retry loop:** Deliberately feed a query that generates wrong SQL on first try. Verify the

Day 5 — End-of-Day Checklist

- LangGraph state machine running with planner → generator → validator nodes
- Agent successfully self-corrects at least 1 query that failed on first attempt
- Retry loop stops at max 3 attempts
- Agent calls tools (graph lookup, RAG retrieval) as part of its reasoning
- Can explain to mentor: what an agent is, the ReAct loop, how self-correction improves accuracy

DAY 6

Validation + API + UI

Package it into a usable application

9:00 - 10:30

LEARN

FastAPI + Streamlit Basics

- **FastAPI quickstart:** Create an endpoint that takes JSON input and returns JSON. Learn: @app.post, Pydantic models, async. (30 min).
- **Streamlit quickstart:** st.text_input, st.button, st.write, st.code. Build a hello-world chat interface. (30 min).

10:30 - 18:00

BUILD

Full Application Stack

- **Build SQL validator:** Using sqlglot — parse SQL into AST, check syntax, validate table/column names against your schema graph. Return structured errors: 'Column X not found, did you mean Y?'
- **Add sandbox executor:** Execute generated SQL on a read-only SQLite copy of your sample DB. Return results or error message. Add timeout (10 sec max).
- **Build FastAPI endpoints:** POST /upload-schema (accepts Excel file), POST /generate-sql (accepts NL query, returns SQL + confidence), GET /schema-info (returns table list).
- **Build Streamlit chat UI:** Text input for NL query, displays generated SQL with syntax highlighting, shows table results below, has a 'thumbs up/down' feedback button.
- **Wire together:** Streamlit UI → FastAPI → LangGraph agent → SQL output. Test the full flow end-to-end.
- **Add guardrails:** Simple check — reject any SQL containing DROP, DELETE, UPDATE, ALTER,

Day 6 — End-of-Day Checklist

- SQL validator catches syntax errors and wrong column names
- FastAPI server running with /generate-sql endpoint
- Streamlit chat UI where user types NL → sees SQL + results
- Full pipeline works: UI → API → Agent → RAG → SLM → Validator → Response
- Guardrails block harmful SQL (DROP, DELETE, etc.)
- Can demo the full app to someone and it works for basic queries

DAY 7 Evaluation + Polish + Demo

Measure accuracy, fix bugs, prepare presentation

9:00 - 10:00

LEARN

Evaluation Metrics

- **Execution Accuracy (EX):** Run both generated and gold-standard SQL, compare result sets. (10 min).
- **Valid SQL Rate (VSR):** Percentage of queries that parse without errors. (5 min).
- **How to present a capstone project:** Problem → Solution → Architecture → Demo → Results →

10:00 - 14:00

BUILD

Evaluation + Bug Fixes

- **Create test suite:** 20-30 NL questions with gold-standard SQL answers. Cover: simple SELECT (5), single JOIN (5), 2-table JOIN (5), 3+ table JOIN (5), aggregation (5), WHERE filters (5).
- **Run evaluation:** Script that runs all 30 queries through your agent, compares results, reports EX and VSR.
- **Fix top failures:** Look at the queries that failed. Are they RAG failures (wrong tables retrieved)? Graph failures (wrong join path)? SLM failures (wrong SQL syntax)? Fix the most impactful category.
- **Add Docker Compose:** Create docker-compose.yml with your FastAPI + Streamlit + SQLite

14:00 - 18:00

POLISH

Demo Prep + Documentation

- **Prepare 3 demo queries:** One simple, one medium (2-table JOIN), one complex (3+ table JOIN with aggregation). Pre-test them — make sure they work flawlessly.
- **Record a backup video:** Screen-record the demo in case of live demo issues.
- **Write README.md:** Project overview, architecture diagram (link your PDF), setup instructions, demo queries.
- **Update capstone PDF:** Add your actual evaluation results (EX score, VSR score) to Section 8.
- **Prepare mentor talking points:** What you built, what you learned, what you'd improve with more

Day 7 — Final Checklist

- Evaluation script runs on 20+ test queries and reports accuracy
- EX accuracy documented (target: 50-70% for a 1-week sprint is excellent)
- 3 demo queries work flawlessly end-to-end
- Docker Compose file packages the full application
- README.md with setup instructions and architecture overview
- Capstone PDF updated with actual results
- Backup demo video recorded
- Can confidently walk mentor through: problem → solution → architecture → demo → results

DAY

Survival Tips for the Sprint

Things that will save you hours

TIP

- **Start with NetworkX, not Neo4j.** NetworkX is a pip install and works in-memory. Neo4j requires Docker, configuration, and Cypher learning. For a 1-week sprint with 5-6 tables, NetworkX is the

TIP

- **If fine-tuning is hard, use few-shot prompting instead.** Load CodeLlama-7B-Instruct via llama-cpp-python (GGUF format, runs on CPU) and inject 3-5 example NL→SQL pairs into the prompt. This is architecturally equivalent — the RAG pipeline is doing the same work. Many

TIP

- **Build the sample database on Day 1 and reuse it all week.** A well-designed 5-6 table schema with realistic data is the foundation everything runs on. Spend 30 extra minutes making it good.

TIP

- **Commit code at the end of every day.** You want 7 clean commits showing daily progress. Your mentor will appreciate seeing the build-up.

TIP

- **50-60% accuracy in 1 week is a great result.** Don't chase 70%+ at the expense of having a working demo. A polished system at 55% accuracy impresses more than a broken system that

TIP

- **Use Claude/ChatGPT as a coding assistant.** When stuck on a specific implementation, ask for help. Focus your learning time on understanding the concepts, not debugging import errors.

*Good luck with your sprint! The goal is a working system you can demo with confidence.
Everything beyond that is bonus. You've got this.*